

Sistemas Distribuidos

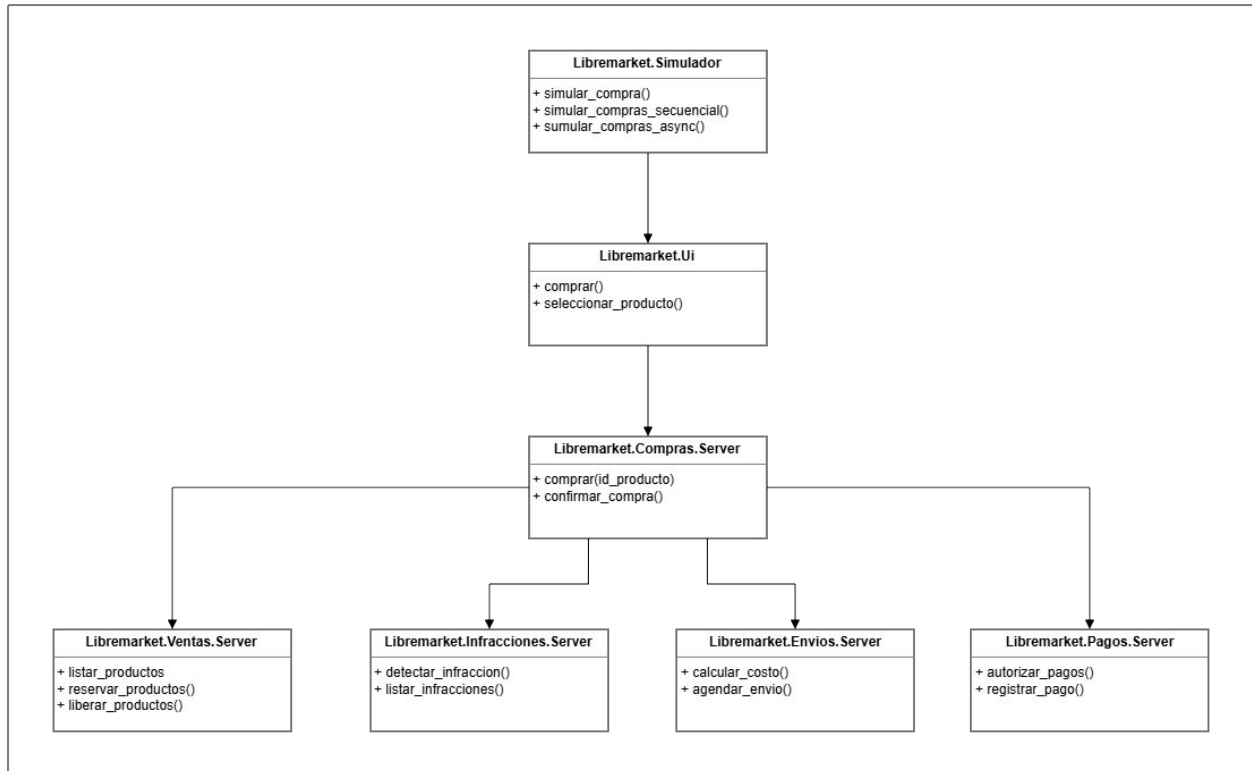
Laboratorio 1



Integrantes del grupo: Agustin Medina, Sebastian Ortega

Fecha de entrega: 5/9/2026

Ejercicio 1:



Ejercicio 2:

Las estructuras de datos seleccionadas para mantener el estado de cada servidor son las siguientes:

El estado de **Ventas.Server** es un mapa que contiene un mapa de productos. Cada producto se identifica mediante su `id_producto`, utilizado como clave, y su valor es otro mapa que contiene los datos del producto, como su nombre y stock disponible.

El estado de **Compras.Server** es un mapa que contiene el próximo identificador de compra y un mapa de compras. Cada compra se almacena utilizando su `id_compra` como clave, y como valor se utiliza otro mapa que contiene los datos de la compra.

El estado de **Pagos.Server** es un mapa donde la clave es el identificador de la compra y el valor representa el resultado de la autorización del pago.

El estado de **Infracciones.Server** es un mapa donde cada compra se identifica mediante su `id_compra` y el valor asociado indica si se detectó o no una infracción.

Ejercicio 3:

Sí, existen problemas de concurrencia en el proceso de compras. Se produce un deadlock en el módulo de Ventas al momento de liberar un producto, ya que, para ejecutar este procedimiento, se requiere que el módulo haya sido notificado tanto de la infracción como del resultado del proceso de autorización del pago.

Sin embargo, esto no ocurre porque, al recibir la notificación de una infracción, el módulo de Compras no realiza la llamada al módulo de Pagos para solicitar la autorización correspondiente. Como consecuencia, el módulo de Ventas queda esperando una notificación que nunca recibe, produciéndose el deadlock.

Ejercicio 5:

En la siguiente imagen se puede ver una compra exitosa, dos compras con infracción y una compra rechazada por falta de stock.

```
iex(nl@6d4621d38bee)4> Libremarket.Ventas.Server.listar_productos()
%{
  1 => %{stock: 2, nombre: "Notebook"},
  2 => %{stock: 8, nombre: "Mouse"},
  3 => %{stock: 4, nombre: "Keyboard"},
  4 => %{stock: 3, nombre: "Monitor"},
  5 => %{stock: 10, nombre: "Auriculares"},
  6 => %{stock: 4, nombre: "Webcam"},
  7 => %{stock: 3, nombre: "Parlante"},
  8 => %{stock: 7, nombre: "Microfono"},
  9 => %{stock: 9, nombre: "Tablet"},
  10 => %{stock: 10, nombre: "Impresora"}
}
iex(nl@6d4621d38bee)5> Simulador.simular_compra(1)
%{
  productos: [1],
  estado_del_producto: {:ok, :productos_reservados},
  forma_de_entrega: {:costo_calculado, 120},
  id_compra: 740,
  infraccion: :no_infraccion,
  medio_de_pago: :transferencia
}
iex(nl@6d4621d38bee)6> Simulador.simular_compra(1)
:infraccion_detectada
iex(nl@6d4621d38bee)7> Simulador.simular_compra(1)
:infraccion_detectada
iex(nl@6d4621d38bee)8> Simulador.simular_compra(1)
%{
  productos: [1],
  estado_del_producto: {:ok, :productos_reservados},
  forma_de_entrega: {:costo_calculado, 120},
  id_compra: 805,
  infraccion: :no_infraccion,
  medio_de_pago: :efectivo
}
iex(nl@6d4621d38bee)9> Simulador.simular_compra(1)
:out_of_stock
```

En la siguiente imagen se puede ver una compra cuyo pago fue rechazado.

```
iex(n1@6d4621d38bee)16> Simulador.simular_compra(2)  
:compra_rechazada
```